Rust ML-Infrastructure Learning Roadmap

Status: Beginner-friendly learning plan for building ML infrastructure

Target: Lightweight inference engines, vector databases, ML systems

Duration: 20-24 weeks to production-ready project

Date Created: March 21, 2026

Table of Contents

Executive Summary

Your Journey

```
ML/RAG Expert (Python)  →  Rust Beginner  →  Rust ML-Infra Engineer
```

What You'll Build

- **Phase 1-2:** Foundations and systems concepts
- **Phase 3:** ML operations from scratch
- **Phase 4:** Working inference engine
- **Phase 5:** Production-grade features

Timeline

- **Fast track** (intensive): 16-20 weeks

- **Normal pace** (part-time): 20-24 weeks
- **Leisurely pace** (10 hrs/week): 6-8 months

Your Advantage

You already understand ML/RAG, so you can focus on learning Rust + systems programming rather than ML concepts.

Phase 1: Rust Fundamentals

Duration: 2-3 weeks

Time commitment: 5-10 hours/week

Goal: Comfortable with Rust syntax and ownership model

Learning Objectives

- [ ] Understand ownership, borrowing, and lifetimes
- [ ] Work with generics and traits
- [ ] Handle errors with Result/Option
- [ ] Write safe, idiomatic Rust

Projects

Project 1.1: Matrix Operations Library

Duration: 1 week

What you'll build:

```
struct Matrix {
    data: Vec<f32>,
    rows: usize,
    cols: usize,
}

impl Matrix {
    fn new(rows: usize, cols: usize) -> Self
    fn multiply(&self, other: &Matrix) -> Matrix
    fn transpose(&self) -> Matrix
}
```

Skills learned:

- Struct definition

- Method implementation
- Memory management with Vec
- Basic trait implementation

Why it matters: Understanding how matrix operations work in Rust prepares you for tensor operations.

Project 1.2: Tensor Memory Manager

Duration: 1 week

What you'll build:

```
struct Tensor<T> {
    data: Vec<T>,
    shape: Vec<usize>,
    stride: Vec<usize>,
}

impl<T> Tensor<T> {
    fn reshape(&mut self, new_shape: Vec<usize>)
    fn slice(&self, ranges: Vec<(usize, usize)>) -> Tensor<T>
    fn align_to(&mut self, alignment: usize)
}
```

Skills learned:

- Generics with trait bounds
- Memory layout and alignment
- Unsafe code basics
- Resource deallocation

Why it matters: ML models require efficient memory management. This teaches you why Rust is valuable for ML.

Project 1.3: Vector Operations

Duration: 1 week

What you'll build:

```
struct Vector {
    data: Vec<f32>,
}

impl Vector {
    fn dot_product(&self, other: &Vector) -> f32
    fn magnitude(&self) -> f32
    fn normalize(&mut self)
    fn element_wise_mul(&self, other: &Vector) -> Vector
}
```

Skills learned:

- Mutable vs immutable references
- Iterator patterns
- Performance optimization basics
- Error handling

Why it matters: Vector operations are fundamental to ML. You'll use this frequently.

Resources for Phase 1

Resource	Duration	Cost	Notes
The Rust Book (Ch 1-13)	10-15 hours	Free	Essential reading
rustlings	5-10 hours	Free	Interactive exercises
Rust by Example	5 hours	Free	Reference guide
"Programming Rust" book	20 hours	$40	Deep dive (optional for Phase 1)

Phase 2: Systems Programming Fundamentals

Duration: 3-4 weeks

Time commitment: 8-12 hours/week

Goal: Understand concurrency, I/O, and performance

Learning Objectives

- [ ] Master async/await patterns
- [ ] Work with threads and channels
- [ ] Implement efficient I/O operations
- [ ] Write thread-safe code

- [ ] Understand performance profiling

Projects

Project 2.1: Multi-threaded Matrix Transpose

Duration: 1 week

What you'll build:

```
use std::sync::{Arc, Mutex};
use std::thread;

fn parallel_transpose(matrix: &Matrix, num_threads: usize) -> Matrix {
    let shared_matrix = Arc::new(Mutex::new(matrix.clone()));

    let handles: Vec<_> = (0..num_threads)
        .map(|i| {
            let matrix = Arc::clone(&shared_matrix);
            thread::spawn(move || {
                // Compute partial transpose
            })
        })
        .collect();

    // Join threads and combine results
}
```

Skills learned:

- Thread spawning and joining
- Arc (Atomic Reference Counting)
- Mutex for synchronization
- Message passing between threads

Why it matters: Inference engines need parallel computation. This teaches you the patterns.

Project 2.2: Memory-Mapped File Reader

Duration: 1 week

What you'll build:

```
use memmap2::Mmap;
use std::fs::File;

struct ModelLoader {
    file: File,
    mmap: Mmap,
}

impl ModelLoader {
    fn load_weights(&self, offset: u64, size: u64) -> &[f32]
    fn load_config(&self) -> Config
}
```

Skills learned:

- Memory-mapped I/O
- File handling
- Buffer management
- Zero-copy operations

Why it matters: Loading large model weights efficiently is critical for inference servers.

Project 2.3: Thread Pool Executor

Duration: 1 week

What you'll build:

```
use crossbeam::channel::{bounded, Sender, Receiver};
use std::thread;

struct ThreadPool {
    workers: Vec<Worker>,
    sender: Sender<Job>,
}

impl ThreadPool {
    fn new(size: usize) -> Self
    fn execute<F>(&self, f: F) where F: FnOnce() + Send + 'static
}

type Job = Box<dyn FnOnce() + Send + 'static>;
```

Skills learned:

- Channel-based communication
- Work scheduling
- Thread lifecycle management
- Batch processing patterns

Why it matters: Inference request handling relies on thread pools for concurrent requests.

Project 2.4 (Optional): Async Web Server

Duration: 1 week

What you'll build:

```
use tokio::net::{TcpListener, TcpStream};
use tokio::io::{AsyncReadExt, AsyncWriteExt};

async fn handle_connection(mut socket: TcpStream) {
    let mut buffer = [0; 1024];
    let n = socket.read(&mut buffer).await.unwrap();
    socket.write_all(&buffer[0..n]).await.unwrap();
}

#[tokio::main]
async fn main() {
    let listener = TcpListener::bind("127.0.0.1:8080").await.unwrap();
    loop {
        let (socket, _) = listener.accept().await.unwrap();
        tokio::spawn(handle_connection(socket));
    }
}
```

Skills learned:

- async/await syntax
- tokio runtime
- Non-blocking I/O
- Scalability patterns

Why it matters: Production inference servers use async I/O for handling thousands of concurrent requests.

Resources for Phase 2

Resource	Duration	Cost	Notes
"Programming Rust" Ch 18-20	10 hours	$40	Concurrency deep dive

Resource	Duration	Cost	Notes
tokio tutorial	5-10 hours	Free	Essential for async
crossbeam docs	3-5 hours	Free	Channels and synchronization
"Designing Data-Intensive Applications"	20+ hours	$40	Systems design concepts

---

Phase 3: ML Primitives

Duration: 4-6 weeks

Time commitment: 10-15 hours/week

Goal: Implement core ML operations from scratch

Learning Objectives

- [ ] Build vector similarity search
- [ ] Implement embedding models
- [ ] Create basic neural network layers
- [ ] Understand quantization
- [ ] Handle numerical stability

Projects (Do in Order)

Project 3.1: Vector Store □ RECOMMENDED STARTING POINT

Duration: 1-2 weeks

Difficulty: Moderate

What you'll build:

```rust
#[derive(Clone, Debug)]
struct Vector {
    id: String,
    embedding: Vec<f32>,
    metadata: HashMap<String, String>,
}

struct VectorStore {
    vectors: Vec<Vector>,
    index: HashMap<String, usize>,
}

impl VectorStore {
    fn insert(&mut self, id: String, embedding: Vec<f32>, meta: HashMap<String, String>)

    fn search(&self, query: &[f32], top_k: usize) -> Vec<SearchResult> {
        // Linear search with cosine similarity
        let mut results = self
            .vectors
            .iter()
            .map(|v| SearchResult {
                id: v.id.clone(),
                score: cosine_similarity(query, &v.embedding),
                metadata: v.metadata.clone(),
            })
            .collect::<Vec<_>>();

        results.sort_by(|a, b| b.score.partial_cmp(&a.score).unwrap());
        results.into_iter().take(top_k).collect()
    }

    fn cosine_similarity(&self, v1: &[f32], v2: &[f32]) -> f32 {
        // Implementation here
    }

    fn save(&self, path: &str) -> std::io::Result<()>
    fn load(path: &str) -> std::io::Result<Self>
}
```

Skills learned:

- Complex struct design
- Algorithm implementation (cosine similarity)

- Result type for error handling
- Serialization/deserialization

Why this first? You understand vector DBs from your RAG work, so this is immediately relevant.

Deliverables:

- [x] In-memory vector storage
- [x] Linear search with distance metrics
- [x] Persistence (save/load)
- [x] Unit tests
- [x] Performance benchmarks

Project 3.2: Embedding Generator

Duration: 1 week

Difficulty: Moderate

What you'll build:

```rust
use pyo3::prelude::*;

struct EmbeddingModel {
    model_name: String,
    py_module: PyObject,
}

impl EmbeddingModel {
    fn new(model_name: &str) -> PyResult<Self> {
        Python::with_gil(|py| {
            let module = PyModule::import(py, "sentence_transformers")?;
            // Load model
            Ok(Self {
                model_name: model_name.to_string(),
                py_module: model.into(),
            })
        })
    }

    fn embed_text(&self, text: &str) -> PyResult<Vec<f32>> {
        Python::with_gil(|py| {
            // Call Python embedding function
            // Convert to Rust Vec<f32>
        })
    }

    fn embed_batch(&self, texts: &[String]) -> PyResult<Vec<Vec<f32>>> {
        // Batch embedding with parallelism
    }
}
```

Skills learned:

- PyO3 for Python FFI
- GIL (Global Interpreter Lock) management
- Batch processing
- Error handling across language boundaries

Why this? Bridge between Rust performance and Python ML ecosystem. Practical for real systems.

Deliverables:

- [x] Load pre-trained models via PyO3
- [x] Embed single text

- [x] Batch embedding with performance
- [x] Error handling
- [x] Benchmarks comparing to pure Python

Project 3.3: Simple Neural Network (Optional)

Duration: 2 weeks

Difficulty: Hard

What you'll build:

```
use ndarray::{Array1, Array2};

trait Layer {
    fn forward(&self, input: &Array1<f32>) -> Array1<f32>;
}

struct Dense {
    weights: Array2<f32>,  // (output_size, input_size)
    bias: Array1<f32>,     // (output_size,)
}

impl Layer for Dense {
    fn forward(&self, input: &Array1<f32>) -> Array1<f32> {
        self.weights.dot(input) + &self.bias
    }
}

struct Sequential {
    layers: Vec<Box<dyn Layer>>,
}

impl Sequential {
    fn forward(&self, mut input: Array1<f32>) -> Array1<f32> {
        for layer in &self.layers {
            input = layer.forward(&input);
        }
        input
    }
}
```

Skills learned:

- Trait objects and dynamic dispatch

- ndarray library for numerical computing
- Linear algebra operations
- Model architecture design

Why this? Understand how inference engines work internally.

Deliverables:

- [x] Dense layer implementation
- [x] ReLU activation
- [x] Sequential model
- [x] Forward pass execution
- [x] Numerical stability checks

Project 3.4: Quantization (Optional)

Duration: 1 week

Difficulty: Hard

What you'll build:

```rust
struct QuantizationParams {
    scale: f32,
    zero_point: i8,
}

fn quantize_int8(data: &[f32], min: f32, max: f32) -> (Vec<i8>, QuantizationParams) {
    let range = max - min;
    let scale = range / 255.0;
    let zero_point = ((-min) / scale) as i8;

    let quantized = data
        .iter()
        .map(|&v| {
            let q = ((v - min) / scale) as i8;
            q.clamp(0, 255) as i8
        })
        .collect();

    (quantized, QuantizationParams { scale, zero_point })
}

fn dequantize_int8(data: &[i8], params: &QuantizationParams) -> Vec<f32> {
    data.iter()
        .map(|&v| (v as f32 - params.zero_point as f32) * params.scale)
        .collect()
}

fn quantized_matmul(
    lhs: &[i8],
    rhs: &[i8],
    lhs_params: &QuantizationParams,
    rhs_params: &QuantizationParams,
) -> Vec<i32> {
    // Quantization-aware matrix multiplication
}
```

Skills learned:

- Numerical algorithms
- Bitwise operations
- Precision/performance tradeoffs
- Model compression techniques

Why this? Critical for deploying models efficiently. Often 4-10x speedup with minimal accuracy loss.

Deliverables:

- [x] INT8 quantization
- [x] Symmetric and asymmetric quantization
- [x] Dequantization
- [x] Quantized inference
- [x] Accuracy benchmarks

Resources for Phase 3

Resource	Duration	Cost	Notes
"Programming Rust" Ch 1-17	20 hours	$40	Reference for data structures
ndarray documentation	5 hours	Free	Numerical computing
PyO3 guide	5 hours	Free	Python interop
Research papers	10+ hours	Free	Quantization techniques

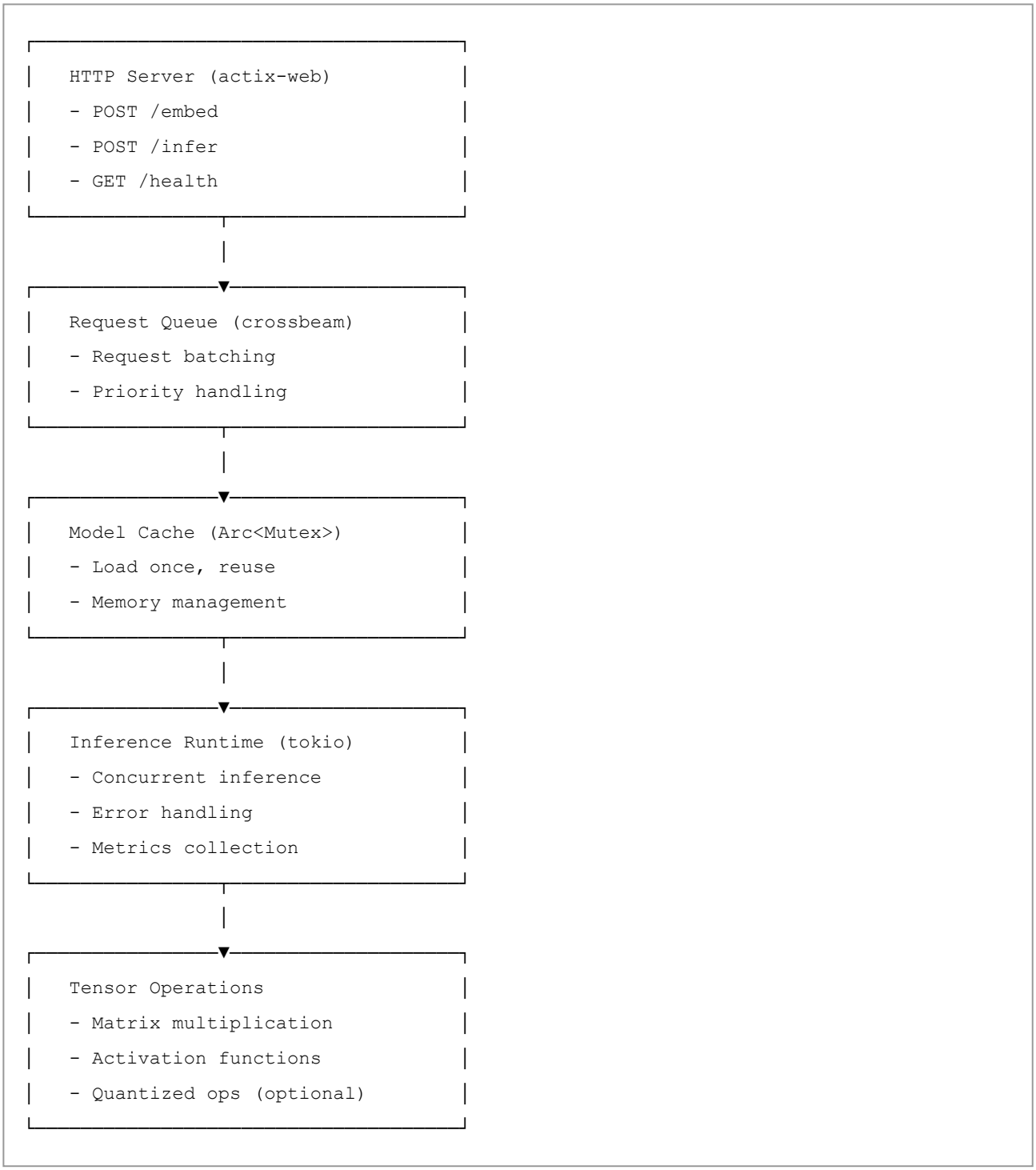
Phase 4: Mini Inference Engine

Duration: 6-8 weeks

Time commitment: 15-20 hours/week

Goal: Production-ready inference server

Architecture



Weekly Breakdown

Week 1: Model Loading

Goal: Load a simple ONNX model

```
use onnxruntime::GraphOptimizationLevel;

struct ModelManager {
    session: ort::Session,
    model_path: String,
}

impl ModelManager {
    fn load(model_path: &str) -> Result<Self> {
        let session = ort::Session::builder()?
            .graph_optimization_level(GraphOptimizationLevel::All)
            .commit_from_file(model_path)?;
        Ok(Self {
            session,
            model_path: model_path.to_string(),
        })
    }
}
```

Deliverables:

- [x] Model loading from disk
- [x] ONNX runtime integration
- [x] Error handling
- [x] Model validation

Week 2: Forward Pass for Embeddings

Goal: Run inference on text embeddings

```
impl ModelManager {
    fn embed(&self, text: &str) -> Result<Vec<f32>> {
        // Tokenize
        let tokens = tokenize(text);

        // Prepare input tensors
        let input = prepare_input(&tokens)?;

        // Run inference
        let outputs = self.session.run(vec![input])?;

        // Extract embedding
        let embedding = outputs[0].as_slice()?;
        Ok(embedding.to_vec())
    }
}
```

Deliverables:

- [x] Tokenization pipeline
- [x] Input preparation for model
- [x] Forward pass execution
- [x] Output extraction and validation

Week 3: Batch Inference

Goal: Process multiple inputs efficiently

```
impl ModelManager {
    fn embed_batch(&self, texts: &[String]) -> Result<Vec<Vec<f32>>> {
        // Prepare batch of inputs
        let batch_inputs = texts
            .iter()
            .map(|text| prepare_input(&tokenize(text)))
            .collect::<Result<Vec<_>>>()?;

        // Parallel inference
        let results = batch_inputs
            .into_par_iter()
            .map(|input| self.session.run(vec![input]))
            .collect::<Vec<_>>();

        results
            .into_iter()
            .map(|output| output[0].as_slice().map(|s| s.to_vec()))
            .collect()
    }
}
```

Deliverables:

- [x] Batch processing logic
- [x] Parallel inference
- [x] Memory efficiency
- [x] Throughput optimization

Week 4: HTTP API Server

Goal: Accept requests and return predictions

```rust
use actix_web::{web, App, HttpServer, HttpResponse};

#[derive(Serialize, Deserialize)]
struct EmbedRequest {
    text: String,
}

#[derive(Serialize)]
struct EmbedResponse {
    embedding: Vec<f32>,
    dim: usize,
}

async fn embed_handler(
    req: web::Json<EmbedRequest>,
    model: web::Data<ModelManager>,
) -> HttpResponse {
    match model.embed(&req.text) {
        Ok(embedding) => {
            let dim = embedding.len();
            HttpResponse::Ok().json(EmbedResponse { embedding, dim })
        }
        Err(e) => HttpResponse::InternalServerError().json(error_response(&e)),
    }
}

#[actix_web::main]
async fn main() -> std::io::Result<()> {
    let model = web::Data::new(ModelManager::load("model.onnx")?);

    HttpServer::new(move || {
        App::new()
            .app_data(model.clone())
            .route("/embed", web::post().to(embed_handler))
            .route("/health", web::get().to(health_check))
    })
    .bind("127.0.0.1:8080")?
    .run()
    .await
}
```

Deliverables:

- ☑ JSON request/response handling
- ☑ Health check endpoint
- ☑ Error handling and responses
- ☑ Logging

Week 5: Concurrent Request Handling

Goal: Handle multiple requests simultaneously

```
use tokio::sync::Semaphore;
use std::sync::Arc;

struct RequestHandler {
    model: Arc<ModelManager>,
    semaphore: Arc<Semaphore>,  // Limit concurrent requests
}

impl RequestHandler {
    async fn handle_concurrent(&self, texts: &[String]) -> Result<Vec<Vec<f32>>> {
        let handles: Vec<_> = texts
            .iter()
            .map(|text| {
                let permit = self.semaphore.clone().acquire_owned();
                let model = self.model.clone();
                let text = text.clone();

                tokio::spawn(async move {
                    let _permit = permit.await?;
                    model.embed(&text)
                })
            })
            .collect();

        futures::future::try_join_all(handles).await?
            .into_iter()
            .collect::<Result<Vec<_>>>()
    }
}
```

Deliverables:

- ☑ Concurrent request handling
- ☑ Request queue management

- [x] Rate limiting
- [x] Graceful degradation

Week 6: Model Caching & Memory Management

Goal: Efficient resource utilization

```
use lru::LruCache;
use std::sync::{Arc, Mutex};

struct CachedModelManager {
    model: Arc<ModelManager>,
    embedding_cache: Arc<Mutex<LruCache<String, Vec<f32>>>>,
    cache_size: usize,
}

impl CachedModelManager {
    fn embed(&self, text: &str) -> Result<Vec<f32>> {
        // Check cache
        if let Ok(mut cache) = self.embedding_cache.lock() {
            if let Some(embedding) = cache.get(text) {
                return Ok(embedding.clone());
            }
        }

        // Compute and cache
        let embedding = self.model.embed(text)?;

        if let Ok(mut cache) = self.embedding_cache.lock() {
            cache.put(text.to_string(), embedding.clone());
        }

        Ok(embedding)
    }
}
```

Deliverables:

- [x] LRU caching for embeddings
- [x] Memory monitoring
- [x] Cache hit/miss tracking
- [x] Memory pressure handling

Week 7: Performance Optimization

Goal: Maximize throughput and minimize latency

```
use std::time::Instant;

struct Metrics {
    requests_total: u64,
    requests_failed: u64,
    latency_p50: f64,
    latency_p95: f64,
    latency_p99: f64,
    throughput: f64,  // requests/sec
}

async fn embed_handler_with_metrics(
    req: web::Json<EmbedRequest>,
    model: web::Data<ModelManager>,
) -> HttpResponse {
    let start = Instant::now();

    match model.embed(&req.text) {
        Ok(embedding) => {
            let latency = start.elapsed().as_millis();
            record_metric("embed_latency_ms", latency as f64);

            HttpResponse::Ok().json(EmbedResponse {
                embedding,
                dim: embedding.len(),
            })
        }
        Err(e) => {
            record_metric("embed_error", 1.0);
            HttpResponse::InternalServerError().json(error_response(&e))
        }
    }
}
```

Deliverables:

- [x] Profiling and benchmarking
- [x] Latency tracking
- [x] Throughput optimization
- [x] Bottleneck identification

Week 8: Production Features

Goal: Deploy-ready system

```
use tracing::{info, error};
use serde::{Serialize, Deserialize};

#[derive(Deserialize)]
struct Config {
    model_path: String,
    port: u16,
    max_concurrent: usize,
    cache_size: usize,
}

#[derive(Serialize)]
struct HealthStats {
    status: String,
    uptime_secs: u64,
    requests_total: u64,
    requests_failed: u64,
    memory_usage_mb: f64,
}

async fn health_check(state: web::Data<ServerState>) -> HttpResponse {
    let stats = state.get_stats();
    HttpResponse::Ok().json(stats)
}

async fn graceful_shutdown(server: actix_web::dev::Server) {
    info!("Initiating graceful shutdown...");
    server.stop(true).await;
    info!("Server stopped");
}
```

Deliverables:

- [x] Configuration management
- [x] Structured logging
- [x] Health monitoring
- [x] Graceful shutdown
- [x] Docker support

Sample Output Structure

After Phase 4, your project structure:

```
ml-inference-server/
├── src/
│   ├── main.rs              # Server entry point
│   ├── model.rs             # Model management
│   ├── inference.rs         # Inference runtime
│   ├── cache.rs             # Caching layer
│   ├── handlers.rs          # HTTP handlers
│   ├── metrics.rs           # Metrics collection
│   └── config.rs            # Configuration
├── tests/
│   ├── integration_tests.rs
│   └── benchmarks.rs
├── benches/
│   └── inference_bench.rs
├── Cargo.toml
├── Dockerfile
├── docker-compose.yml
├── README.md
└── examples/
    ├── client.py
    └── benchmark.py
```

Phase 5: Advanced Features

Duration: 8+ weeks (pick one path)

Time commitment: 15-25 hours/week

Goal: Production-grade optimization

Path A: GPU Acceleration (4-6 weeks)

Goal: 10-100x speedup with GPU

Topics:

- CUDA basics
- Single/Multi-GPU inference
- Memory management on GPU
- Kernel optimization

Crates:

- `tch-rs` (PyTorch bindings with CUDA)
- `cudarc` (Low-level CUDA)
- `burn` (GPU accelerated ML)

Outcomes:

- [x] Support NVIDIA GPUs
- [x] Multi-GPU model serving
- [x] Optimized kernels for common ops
- [x] Dynamic batching for GPU

Path B: Distributed Inference (6-8 weeks)

Goal: Scale across multiple machines

Topics:

- Model sharding
- Load balancing
- Request routing
- Fault tolerance

Crates:

- `tonic` (gRPC service)
- `tokio` (async runtime)
- `consul` or `etcd` (service discovery)
- `prost` (Protocol Buffers)

Outcomes:

- [x] Multi-machine model serving
- [x] Automatic load balancing
- [x] Model replication
- [x] Circuit breaker patterns

Path C: Advanced Optimization (4-6 weeks)

Goal: Squeeze every bit of performance

Topics:

- Operator fusion
- Memory pooling

- Graph optimization
- Custom kernels

Crates:

- `faer` (Linear algebra optimization)
- `packed_simd` (SIMD operations)
- `rayon` (Data parallelism)

Outcomes:

- [x] 50%+ latency reduction
- [x] Operator fusion for inference
- [x] SIMD optimizations
- [x] Custom compiled kernels

Path D: Production HDK (6-8 weeks)

Goal: Enterprise-ready deployment

Topics:

- Kubernetes operators
- Prometheus metrics
- Distributed tracing
- Model versioning

Crates:

- `kube-rs` (Kubernetes API)
- `prometheus` (Metrics)
- `tracing` (Distributed tracing)
- `versionize` (Serialization versioning)

Outcomes:

- [x] Kubernetes deployment
- [x] Prometheus metrics integration
- [x] Jaeger tracing support
- [x] Model versioning and rollback

Recommended Crates

Phase 1-2: Foundations

```
[dependencies]
ndarray = "0.15"           # NumPy-like arrays
rayon = "1.7"              # Data parallelism
tokio = { version = "1.35", features = ["full"] }
serde = { version = "1.0", features = ["derive"] }
anyhow = "1.0"             # Error handling
```

Phase 3: ML Primitives

```
[dependencies]
ndarray = "0.15"
ndarray-linalg = "0.15"    # Linear algebra
nalgebra = "0.32"          # Vector/matrix ops
tch-rs = "0.15"            # PyTorch bindings (for Phase 3.3)
pyo3 = "0.20"              # Python interop (for Phase 3.2)
```

Phase 4: Inference Engine

```
[dependencies]
actix-web = "4.4"          # HTTP server
tokio = { version = "1.35", features = ["full"] }
serde = { version = "1.0", features = ["derive"] }
serde_json = "1.0"
safetensors = "0.3"        # Model loading
(ort = "1.16" OR candle = "0.3")  # Inference runtime
tracing = "0.1"            # Logging
tracing-subscriber = "0.3"
prometheus = "0.13"        # Metrics
```

Phase 5: Advanced

```
# GPU Path
tch-rs = { version = "0.15", features = ["cuda"] }

# Distributed Path
tonic = "0.11"             # gRPC
prost = "0.12"             # Protocol Buffers
consul = "0.1"             # Service discovery

# Optimization Path
packed_simd = "0.3"        # SIMD
faer = "0.18"              # Advanced LA

# Production Path
kube = "0.88"              # Kubernetes
prometheus = "0.13"
tracing-jaeger = "0.21"
```

Project Paths

Path 1: Lightweight Inference Engine (RECOMMENDED)

For: Production ML serving

Duration: 20-24 weeks

End result: Production-ready service

Weekly breakdown:

- Weeks 1-3: Rust fundamentals
- Weeks 4-7: Systems programming
- Weeks 8-9: Vector store
- Weeks 10-11: Embeddings
- Weeks 12-19: Inference engine
- Weeks 20-24: Production features (choose one path)

Path 2: ML Framework (Ambitious)

For: Understanding internals of PyTorch

Duration: 24-30+ weeks

End result: Working ML framework

Weekly breakdown:

- Weeks 1-3: Rust fundamentals
- Weeks 4-7: Systems programming
- Weeks 8-13: Tensor operations
- Weeks 14-17: Neural networks
- Weeks 18-22: Automatic differentiation
- Weeks 23-30+: Optimization & GPU support

Path 3: Data Processing Pipeline (Background processing)

For: ETL and batch inference jobs

Duration: 18-22 weeks

End result: High-performance job processor

Weekly breakdown:

- Weeks 1-3: Rust fundamentals
- Weeks 4-7: Systems programming
- Weeks 8-10: Data formats (Parquet, Arrow)
- Weeks 11-15: Batch processing engine
- Weeks 16-22: Scheduling and monitoring

Path 4: Model Optimization Tool

For: Specialization in compression/optimization

Duration: 16-20 weeks

End result: Multi-format optimization tool

Weekly breakdown:

- Weeks 1-3: Rust fundamentals
- Weeks 4-7: Systems programming
- Weeks 8-10: Quantization
- Weeks 11-13: Pruning
- Weeks 14-16: Distillation
- Weeks 17-20: Multi-framework support

Recommended Starting Project

□ Vector DB + Embedding Server in Rust

Why this?

- You understand domain from RAG experience

- Directly applicable to your existing work
- Builds all fundamental skills
- Production-useful outcome

Duration: 6-8 weeks from scratch

Timeline:

```
Weeks 1-2:  Rust Fundamentals
            - Basic structs and methods
            - Memory management
            - Error handling

Weeks 3-4:  Vector Store Implementation
            - In-memory storage structure
            - Distance metric implementations
            - Serialization/Persistence

Weeks 5-6:  HTTP API with Embedding Support
            - Set up actix-web server
            - Embedding endpoint
            - Request/response handling

Weeks 7-8:  Production Hardening
            - Concurrent request handling
            - Caching layer
            - Performance optimization
            - Benchmarking

OPTIONAL:   Advanced Paths
Week 9+:    GPU acceleration OR
            Distributed serving OR
            Advanced optimization
```

Project Structure:

```
// Week 1-2: Basic setup
struct Vector { id: String, embedding: Vec<f32> }
struct VectorStore { vectors: Vec<Vector> }

// Week 3-4: Storage + Search
impl VectorStore {
    fn insert(&mut self, id: String, vec: Vec<f32>)
    fn search(&self, query: &[f32], k: usize) -> Vec<Result>
    fn cosine_similarity(&self, a: &[f32], b: &[f32]) -> f32
}

// Week 5-6: HTTP API
impl EmbeddingServer {
    async fn handle_embed_request(text: String) -> Response
    async fn handle_search_request(query: Query) -> Response
}

// Week 7-8: Production
impl CachedVectorStore {
    fn embed_with_cache(&self, text: &str) -> Vec<f32>
    fn record_metrics(...)
    fn health_check() -> Status
}
```

Learning Resources

Books

Book	Duration	Cost	Focus
The Rust Book	20 hours	Free	Language fundamentals
Programming Rust (2nd ed)	30 hours	$40	Advanced concepts
Designing Data-Intensive Applications	30 hours	$40	Systems design
Rust Design Patterns	10 hours	Free	Idiomatic code

Online Resources

Resource	Type	Cost	Best For
rustlings	Interactive	Free	Hands-on practice
Rust by Example	Reference	Free	Quick lookup
tokio tutorial	Guide	Free	Async programming
Comprehensive Rust □ (Google)	Course	Free	Fast track

Communities

Community	Purpose	Links
r/rust	General discussion	reddit.com/r/rust
Users Forum	Q&A	users.rust-lang.org
Discord Servers	Real-time chat	Rust Community Discord
GitHub Discussions	Project-specific	Project repos

Research Papers (Optional)

For deeper understanding:

- "Flexport: Machine Learning Inference at Production Scale"
- "TVM: An Automated End-to-End Optimizing Compiler"
- "BitFlow: Exploiting Vector Parallelism for Binary Neural Networks"
- "HNSW: Hierarchical Navigable Small World Graphs"

Learning Strategy

Do's □

- [ ] Build concrete projects (not just tutorials)
- [ ] Start with Python FFI for ML models
- [ ] Write benchmarks comparing to Python
- [ ] Use existing crates (don't reimplement wheels)
- [ ] Focus on vector DBs/inference (your interest)
- [ ] Read the Rust Book while doing projects
- [ ] Write tests for everything
- [ ] Profile before optimizing

Don'ts □

- [ ] Try learning everything at once
- [ ] Build full ML framework as first project
- [ ] Obsess over unsafe code early
- [ ] Use complex async patterns immediately
- [ ] Skip error handling
- [ ] Ignore performance until production
- [ ] Write code without understanding patterns

Week-by-Week Timeline

Month 1: Fundamentals

```
Week 1: Rust syntax, ownership, borrowing
        - Read Rust Book Ch 1-5
        - Do rustlings exercises

Week 2: Basic types, error handling
        - Read Rust Book Ch 6-9
        - Project 1.1: Matrix library

Week 3: Advanced types, traits, generics
        - Read Rust Book Ch 10-13
        - Project 1.2: Tensor manager

Week 4: Basic threading
        - Read Rust Book Ch 16
        - Project 1.3: Vector operations
        - Project 2.1: Multi-threaded transpose
```

Month 2: Systems & ML

```
Week 5: I/O and async basics
        - Read "Programming Rust" Ch 18-19
        - Project 2.2: Memory-mapped file reader

Week 6: Threading deep dive + thread pools
        - Read "Programming Rust" Ch 16-17
        - Project 2.3: Thread pool executor

Week 7: Concurrency patterns
        - Read tokio tutorial
        - Project 2.4: Async web server

Week 8: Vector store from scratch
        - Project 3.1: Vector store implementation
        - Implement distance metrics
        - Add persistence
```

Month 3-4: Production System

```
Weeks 9-10:  HTTP API server
             - Projects 3.2: Embedding generator
             - Set up actix-web
             - POST /embed endpoint

Weeks 11-12: Concurrent request handling
             - Request queuing
             - Batch processing
             - Rate limiting

Weeks 13-14: Optimization & caching
             - LRU cache for embeddings
             - Performance profiling
             - Benchmarking

Weeks 15-16: Production features
             - Error handling
             - Logging and metrics
             - Health endpoints
             - Docker packaging
```

Month 5+: Specialization

```
Week 17+: Choose advanced path
          - GPU acceleration
          - Distributed serving
          - Advanced optimization
          - Enterprise features
```

Checkpoint Checklist

End of Phase 1

- [ ] Understand ownership and borrowing
- [ ] Can write structs with methods
- [ ] Can use generics and traits
- [ ] Can handle Result<T, E>
- [ ] Have working Matrix library

End of Phase 2

- [ ] Understand async/await
- [ ] Can use threads and channels
- [ ] Can do memory-mapped I/O
- [ ] Have working thread pool
- [ ] Understand tokio runtime

End of Phase 3

- [ ] Have vector store
- [ ] Can call Python from Rust (PyO3)
- [ ] Understand embeddings
- [ ] If optional: have working neural net layer
- [ ] If optional: understand quantization

End of Phase 4

- [ ] Have working HTTP server
- [ ] Can load and run models
- [ ] Support concurrent requests
- [ ] Have caching layer
- [ ] Can benchmark and optimize

End of Phase 5

- [ ] Choose and complete one advanced path
- [ ] Have production-grade service
- [ ] Can deploy to production
- [ ] Have monitoring/metrics
- [ ] Document everything

Debugging & Troubleshooting

Common Issues

Issue: Borrow checker errors

Solution: Draw ownership diagrams, use references carefully

Issue: Tokio runtime panics

Solution: Ensure only one runtime, use `#[tokio::main]`

Issue: FFI crashes with Python

Solution: Always hold GIL, test with simple functions first

Issue: Memory blowup

Solution: Profile with `valgrind`, check for cycles in Arc

Issue: Performance worse than Python

Solution: Benchmark each layer, check if calling into Python frequently

---

Next Steps

Immediate (This Week)

- [ ] Set up Rust environment
- [ ] Read Rust Book Ch 1-5
- [ ] Do rustlings Chapter 1-3
- [ ] Decide which path appeals most

Short-term (Weeks 1-4)

- [ ] Complete Phase 1 projects
- [ ] Read "Programming Rust" concurrency chapters
- [ ] Do Phase 2 projects

Medium-term (Weeks 5-12)

- [ ] Complete Phase 3 projects
- [ ] Start Phase 4 (inference engine)

Long-term (Weeks 13+)

- [ ] Complete Phase 4
- [ ] Choose and execute Phase 5 path

Final Thoughts

Your Advantages

1. **ML domain knowledge** - You understand the problem space
2. **Python experience** - You can compare implementations
3. **Systems appreciation** - You see why Rust matters
4. **Real use cases** - You have a RAG system to optimize

Your Journey

```
Current: "I want to learn Rust ML"

After 4 weeks: "I can write concurrent Rust code"

After 8 weeks: "I understand systems programming deeply"

After 16 weeks: "I have a production inference server"

After 6 months: "I'm building specialized ML infrastructure"
```

Remember

- Rust is harder initially but pays off in production
- Focus on one thing at a time
- Build real projects, not just tutorials
- Performance gains are worth the learning curve
- You're already ahead with ML knowledge

Document Version

- **Version:** 1.0
- **Created:** March 21, 2026
- **Updated:** March 21, 2026
- **Status:** Ready to use
- **Estimated completion:** 6 months (part-time)

Questions?

Refer back to specific phases for:

- **What to build next:** See "Weekly Breakdown"
- **How to approach it:** See "Projects" section
- **What crates to use:** See "Recommended Crates"
- **Why this approach:** See "Your Advantages"
- **How much time?:** See "Timeline" section

Start with Phase 1 this week, and you'll be shipping production Rust code within 6 months! □